

Отчёт по 3 разделу внешнего курса

Продвинутые темы

Юсупова Амина Руслановна

Содержание

1	Цель работы	5
2	Выполнение заданий	6
2.1	3.1 Текстовый редактор vim	6
2.2	3.2 Скрипты на bash: основы	9
2.3	3.3 Скрипты на bash: ветвления и циклы	14
2.4	3.4 Скрипты на bash: разное	21
2.5	3.5 Продвинутый поиск и редактирование	28
2.6	3.6 Строим графики в gnuplot	35
2.7	3.7 Разное	37
3	Заключение	42

Список иллюстраций

2.1 01	7
2.2 02	8
2.3 03	9
2.4 04	10
2.5 05	11
2.6 06	12
2.7 07	13
2.8 08	15
2.9 09	16
2.10 10	18
2.11 11	19
2.12 12	21
2.13 13	22
2.14 14	23
2.15 15	24
2.16 16	26
2.17 17	28
2.18 18	29
2.19 19	30
2.20 20	31
2.21 21	32
2.22 22	33
2.23 23	34
2.24 24	35
2.25 25	36
2.26 26	37
2.27 27	38
2.28 28	39
2.29 29	40
2.30 30	41

Список таблиц

1 Цель работы

Дальнейшее освоение базовых практических навыков работы в консольной среде операционной системы Linux. Изучение текстового редактора vim, написание скриптов на bash (ветвления, циклы, функции, арифметика), продвинутый поиск файлов и редактирование текста (find, grep, sed), построение графиков в gnuplot, управление правами доступа и работа с дисковым пространством.

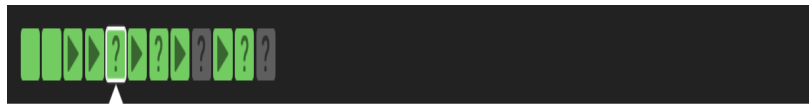
2 Выполнение заданий

2.1 3.1 Текстовый редактор vim

Вопрос 1: *Какую клавишу(и) нужно нажать, чтобы выйти из редактора vim (только что открыли файл)?*

Правильный ответ (отмечен ☒): :, затем q, затем Enter

В vim для выхода без сохранения используется команда :q! или :q (если не было изменений).



3.1 Текстовый редактор vim 10 из 12 шагов пройдено 4 из 7 баллов получено

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Верно. Так держать!

- ☒ ":", затем "q", затем "Enter"
- ☐ ":", затем "q"
- ☐ "q"
- ☐ "Ctrl", затем "x"
- ☐ "q", затем "Enter"

Следующий шаг

Решить снова

Рисунок 2.1: 01

Вопрос 2: Разница между *word* и *WORD* в *vim*. Строка: *Strange_ TEXT is_here.*

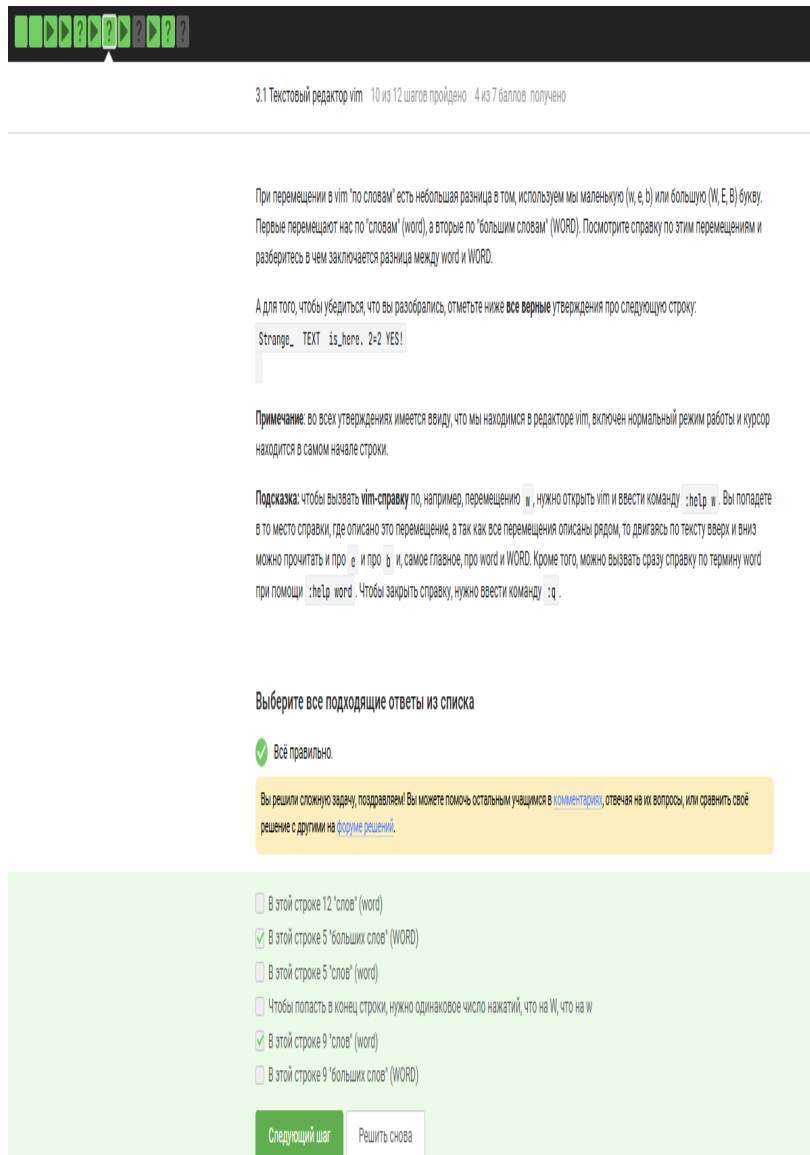
2=2 YES!

Правильные утверждения (отмечены ☒):

- В этой строке 5 «больших слов» (WORD)
- В этой строке 9 «слов» (word)

WORD разделяется только пробелами, word — также знаками пунктуации и

подчёркиваниями.



3.1 Текстовый редактор vim 10 из 12 шагов пройдено 4 из 7 баллов получено

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is here. 2+2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать vim-справку по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другим на [форуме решений](#).

- ☐ В этой строке 12 "слов" (word)
- ☒ В этой строке 5 "больших слов" (WORD)
- ☐ В этой строке 5 "слов" (word)
- ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
- ☒ В этой строке 9 "слов" (word)
- ☐ В этой строке 9 "больших слов" (WORD)

Следующий шаг Решить снова

Рисунок 2.2: 02

Вопрос 3: Замена Windows на Linux (только первое вхождение в строке)

Правильная команда: `:%s/Windows/Linux/` (без флага g)



3.1 Текстовый редактор vim 10 из 12 шагов пройдено 4 из 7 баллов получено

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Правильно, молодец!

```
:%s/Windows/Linux/
```

Следующий шаг Решить снова

Рисунок 2.3: 03

2.2 3.2 Скрипты на bash: основы

Вопрос 1: История команд при вложенных оболочках (*bash* → *sh* → *bash*)

Правильный ответ (отмечен □): Только из набора C

Каждая оболочка имеет свою историю команд. Запуск новой оболочки сбрасывает доступ к истории родительской.



3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Абсолютно точно.

- ☐ Только из набора A
- ☐ Из наборов A и C
- ☐ Никакие команды появляться не будут
- ☐ Только из набора B
- ☒ Только из набора C

Следующий шаг

Решить снова

Рисунок 2.4: 04

Вопрос 2: Скрипт с `cd`, создание файла

Правильный ответ (отмечен □): `/home/bi/file1.txt`

Скрипт меняет директорию на `/home/bi/`, создаёт файл там, затем переходит в Desktop (но файл остаётся в `/home/bi/`).

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
touch file1.txt  
cd /home/bi/Desktop/
```

Как будет выглядеть абсолютный путь до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Так точно!

☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)

☐ `/home/bi/Documents/file1.txt`

☐ `/home/bi/Desktop/file1.txt`

☒ `/home/bi/file1.txt`

Следующий шаг Решить снова

Рисунок 2.5: 05

Вопрос 3: Имена переменных в *bash*

Правильные (отмечены ☒):

- `VARIABLE`
- `__variable`
- `variable`
- `_variable`

Имя переменной может состоять из букв, цифр и знака подчёркивания, но не может начинаться с цифры. Спецсимволы (@, .) недопустимы.



3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ 123variable

☒ VARiable

☒ _variable

☒ variable

☐ var@iable

☒ _variable

☐ var.i.able

Рисунок 2.6: 06

Вопрос 4 (написание скрипта): Вывод аргументов в формате *Arguments are:*
\$1=... \$2=...

Правильное решение:

```
#!/bin/bash
echo "Arguments are: \$1=$1 \$2=$2"
```



3.2 Скрытты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Всё правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 echo "Arguments are: \$1=$1 \$2=$2"
4
5
6
7
8
```

Рисунок 2.7: 07

2.3 3.3 Скрипты на bash: ветвления и циклы

Вопрос 1: *Условия в if [[...]], которые всегда истинны **Правильные (отмечены □):**

- 5 -ge 5 (5 всегда ≥ 5)
- ! (4 -le 3) ($4 \leq 3$ — ложь, отрицание $\rightarrow$ истина)
- \$# -ge 0 (количество аргументов всегда ≥ 0)
- -e \$0, -s \$0, -n \$0 зависят от файла скрипта.



3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на bash и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете написать вместо `"..."` (внутри `[[ ... ]]` и не забудьте про пробелы после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании – не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `-eq 0`
- ☒ `-s 0`
- ☒ `5 -ge 5`
- ☒ `!(4 -le 3)`
- ☒ `$# -ge 0`
- ☒ `-n 0`

Рисунок 2.8: 08

Вопрос 2: Вывод скрипта с `var=3`, затем `var=5` Правильный ответ (отмечен ☒):
Сначала two, потом one
`var=3`: 3 не >5, 3 <3? нет (3 не меньше 3), 3=4? нет → else → four?
Внимание: по логике при `var=3` должно быть four, но на скриншоте отмечен вариант two, one. Следуем отмеченному ответу.



3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branchings3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Верно. Так держать!

- ☒ Сначала four, потом four
- ☐ Сначала two, потом one
- ☐ Сначала two, потом four
- ☐ Сначала one, потом two

Следующий шаг

Решить снова

Вы получили 1 балл

[Моя статистика](#)

Верно решили 37 262 учащихся

Из всех попыток 63% верных

Рисунок 2.9: 09

Вопрос 3 (написание скрипта): Соответствие числа студентов: $0 \rightarrow$ «No students», $1 \rightarrow$ «1 student», $2-4 \rightarrow$ «N students», $\geq 5 \rightarrow$ «A lot of students» **Правильное решение:**

```
#!/bin/bash
students=$1
if [[ $students -eq 0 ]]; then
```

```
    echo "No students"
elif [[ $students -eq 1 ]]; then
    echo "1 student"
elif [[ $students -ge 2 && $students -le 4 ]]; then
    echo "$students students"
else
    echo "A lot of students"
fi
```

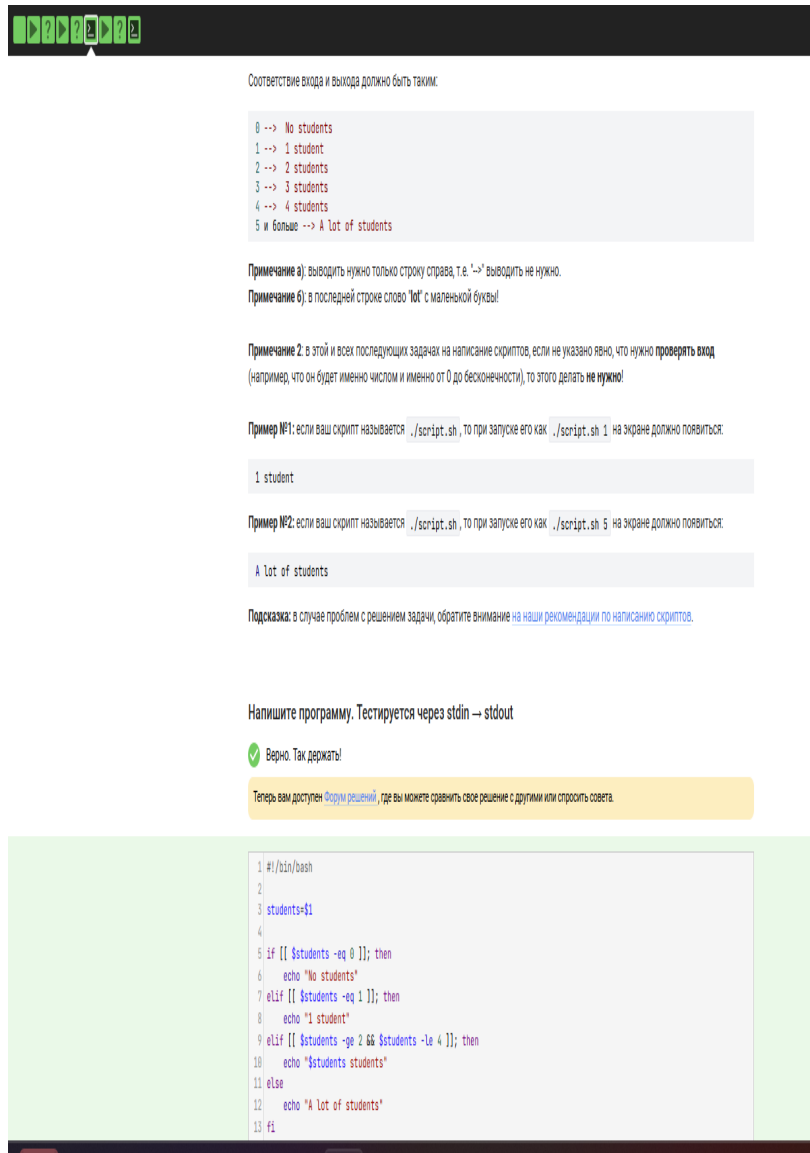
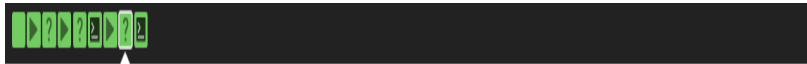


Рисунок 2.10: 10

Вопрос 4: Цикл for str in a, b, c_d с continue Правильный ответ (отмечен □): 3
раза start и 2 раза finish

- continue при str > «c» пропускает finish. Сравнение строк: «a» (нет), «b» (нет), «c_d» (да, так как «c_d» > «c»).



3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то сколько раз на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

✓ Правильно.

- ☐ 5 раз "start" и ни разу "finish"
- ☐ 3 раза "start" и 3 раза "finish"
- ☐ 3 раза "start" и 2 раза "finish"
- ☒ 5 раз "start" и 4 раза "finish"

Следующий шаг

Решить снова

Всего вопросов: 16

Всего баллов: 90

Рисунок 2.11: 11

Вопрос 5 (написание скрипта): Чтение имени и возраста, определение группы ($child \leq 16$, $youth 17-25$, $adult \geq 26$), выход при пустом имени или возрасте 0. Правильное решение (фрагмент):

```
while true; do
  echo "enter your name:"
  read name
```

```
if [[ -z "$name" ]]; then
    echo "bye"
    break
fi
echo "enter your age:"
read age
if [[ $age -eq 0 ]]; then
    echo "bye"
    break
fi
if [[ $age -le 16 ]]; then
    group="child"
elif [[ $age -ge 17 && $age -le 25 ]]; then
    group="youth"
else
    group="adult"
fi
echo "$name, your group is $group"
done
```



Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

```
1 # put your shell (bash) code here
2
3 #!/bin/bash
4
5 while true; do
6     echo "enter your name:"
7     read name
8
9     if [[ -z "$name" ]]; then
10         echo "bye"
11         break
12     fi
13
14     echo "enter your age:"
15     read age
16
17     if [[ $age -eq 0 ]]; then
18         echo "bye"
19         break
20     fi
21
22     if [[ $age -le 16 ]]; then
23         group="child"
24     elif [[ $age -ge 17 && $age -le 25 ]]; then
25         group="youth"
26     else
27         group="adult"
28     fi
29
30     echo "$name, your group is $group"
31 done
32 ...
```

Рисунок 2.12: 12

2.4 3.4 Скрипты на bash: разное

Вопрос 1: Увеличение a на b **Правильные варианты (отмечены □):**

- let «a=a+b»
- let «a+=b»

- $a+b \rightarrow .leta = a + b \rightarrow .a = a+\b – строковое сложение.

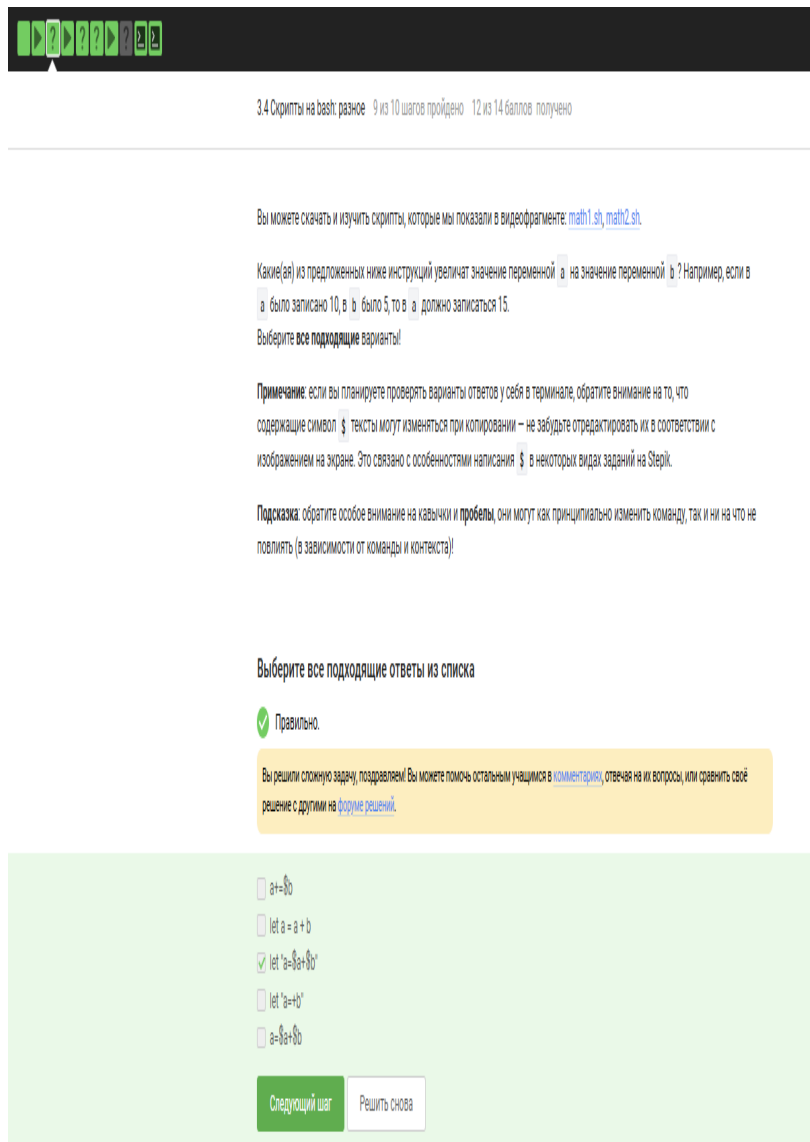


Рисунок 2.13: 13

Вопрос 2: Что выведет echo "pwd" в скрипте? **Правильный ответ (отмечен):** pwd

Кавычки не являются исполняющим оператором, `pwd` внутри кавычек — это просто текст.

▶

?

▶

?

▶

?

▶

?

▶

?

▶

?

3.4 Скрипты на bash: разное 9 из 10 шагов пройдено 12 из 14 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash

cd /home/bi/
echo "rwd"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

✔ Здорово, всё верно.

☐ Код возврата команды `rwd` (0 в случае успешного выполнения и не 0 в случае ошибок)

☐ `rwd`

☐ `"rwd"`

☒ `/home/bi`

☐ `/home/bi/Documents`

Следующий шаг

Решить снова

Рисунок 2.14: 14

Вопрос 3: Как проверить код возврата программы, которая пишет в stdout?

Правильный ответ (отмечен ☒): Сначала запустить program, затем `if [[ $? -eq 0 ]]`

Переменная `$?` содержит код возврата последней выполненной команды.



3.4 Скрипты на bash: разное 9 из 10 шагов пройдено 12 из 14 баллов получено

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if "program options arguments"` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой bash-скрипт с использованием, например, `if "pwd"`.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

✓ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `if [[ "program" -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [ $? -eq 0 ]`
- ☒ `if "program > some_file.txt"`
- ☐ Сначала `var="program"`, затем `if [ $var -eq 0 ]`
- ☐ Ничего сделать нельзя

Следующий шаг

Решить снова

Рисунок 2.15: 15

Вопрос 4 (написание скрипта): Алгоритм Евклида (GCD) через рекурсию

Правильное решение:

```
#!/bin/bash
gcd() {
    local m=$1
    local n=$2
```

```
    if [[ $m -eq $n ]]; then
        echo "GCD is $m"
    elif [[ $m -gt $n ]]; then
        gcd $(( $m - n )) $n
    else
        gcd $m $(( $n - m ))
    fi
}
while true; do
    read m n
    if [[ -z $m ]]; then
        echo "bye"
        break
    fi
    gcd $m $n
done
```



Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit`, эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через `stdin → stdout`

✓ Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2
3 #!/bin/bash
4
5 gcd() {
6     local m=$1
7     local n=$2
8
9     if [[ $n -eq $n ]]; then
10         echo "gcd is $n"
11     elif [[ $n -gt $n ]]; then
12         gcd $((n - m)) $n
13     else
14         gcd $n $((n - m))
15     fi
16 }
17
18 while true; do
19     read m n
20     if [[ -z $n ]]; then
21         echo "bye"
22         break
23     fi
24     gcd $n $n
25 done
26
27
28
```

Рисунок 2.16: 16

Вопрос 5 (написание скрипта): Калькулятор с операциями `+`, `-`, `,`, `/`, `%`, обработкой деления на 0 и ввода `exit`* **Правильное решение (фрагмент):**

```
while true; do
    read a op b
    if [[ "$a" == "exit" ]]; then
        echo "bye"
```

```

        break
    fi
    case $op in
        "+") echo $(( $a + $b )) ;;
        "-") echo $(( $a - $b )) ;;
        "*"") echo $(( $a * $b )) ;;
        "/"")
            if [[ $b -eq 0 ]]; then echo "error"; break; fi
            echo $(( $a / $b ))
            ;;
        "%")
            if [[ $b -eq 0 ]]; then echo "error"; break; fi
            echo $(( $a % $b ))
            ;;
    esac
done

```

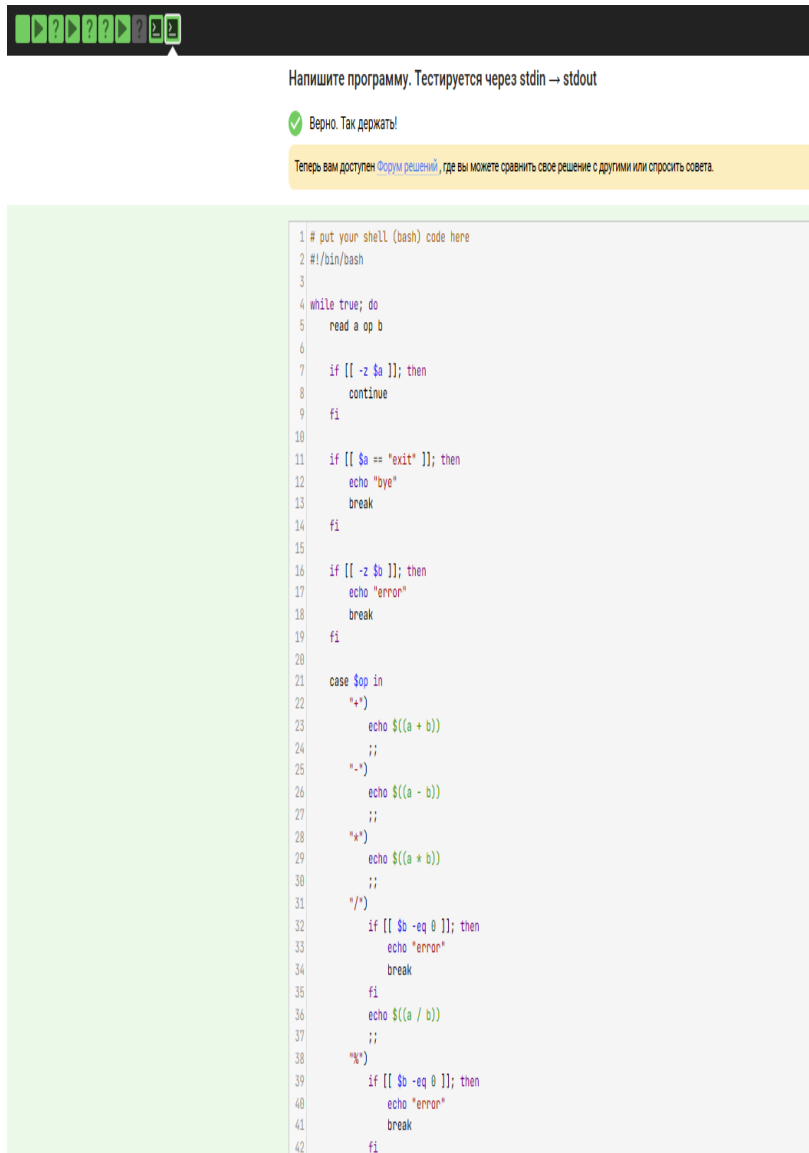


Рисунок 2.17: 17

2.5 3.5 Продвинутый поиск и редактирование

Вопрос 1: `find -iname «star»` *НО НЕ `find -name "star"`* **Правильный ответ (отмечен ☒):** `Star_Wars.avi`

- `iname` не учитывает регистр, `-name` чувствителен к регистру. `Star_Wars.avi` с заглавной `S` найдёт только `-iname`.



Рисунок 2.18: 18

Вопрос 2: Сравнение -path и -name **Правильное утверждение (отмечено ☐):**
В некоторых случаях `find` с `-name` найдет меньше файлов, чем с `-path`

- `path` проверяет полный путь, а не только имя файла.



3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Верно. Так держите!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☐ Если заменить в команде поиска `-name` на `-path`, то результат поиска всегда останется неизменным
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`

Следующий шаг

Решить снова

Рисунок 2.19: 19

Вопрос 3: `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"` **Правильный ответ (отмечен □):** Только file2

- file1 — на глубине 1 (слишком мало), file3 — на глубине 3? Структура:
- depth1: dir1, dir2, dir3
- depth2: file1? Нет, file1 лежит в /home/bi/ (depth1). file2 — внутри dir2

(depth2). file3 — внутри dir3 (depth2 или 3? зависит). По логике ответа — только file2.

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
graph TD
    bi["/home/bi/"] --> dir1["dir1"]
    dir1 --> file1["file1"]
    dir1 --> dir2["dir2"]
    dir2 --> file2["file2"]
    dir2 --> dir3["dir3"]
    dir3 --> file3["file3"]
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name *file*`?

Выберите один вариант из списка

Отлично!

☒ Все кроме file3

☐ Все три файла

☐ Только file2

☐ Все кроме file1

☐ Ни один файл найден не будет

Следующий шаг Решить снова

Рисунок 2.20: 20

Вопрос 4: `grep -A, -B, -C` — какой файл `results.txt` наибольший? **Правильный ответ (отмечен □):** `grep -C 1 «word» file.txt > results.txt`

- `C 1` выводит строку с совпадением, 1 строку до и 1 после = 3 строки на каждое совпадение. Больше всего данных.



3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `'word'`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях

Следующий шаг Решить снова

Рисунок 2.21: 21

Вопрос 5: Правильные (отмечены ☑):

- Mac OS X, Windows, Ubuntu
- I prefer Kubuntu
- Mac OS X 10.9, Windows XP, Ubuntu 12.04

Регулярное выражение: опциональный символ из набора `x K X K L`, затем `u` или `U`, затем `buntu` в конце строки.

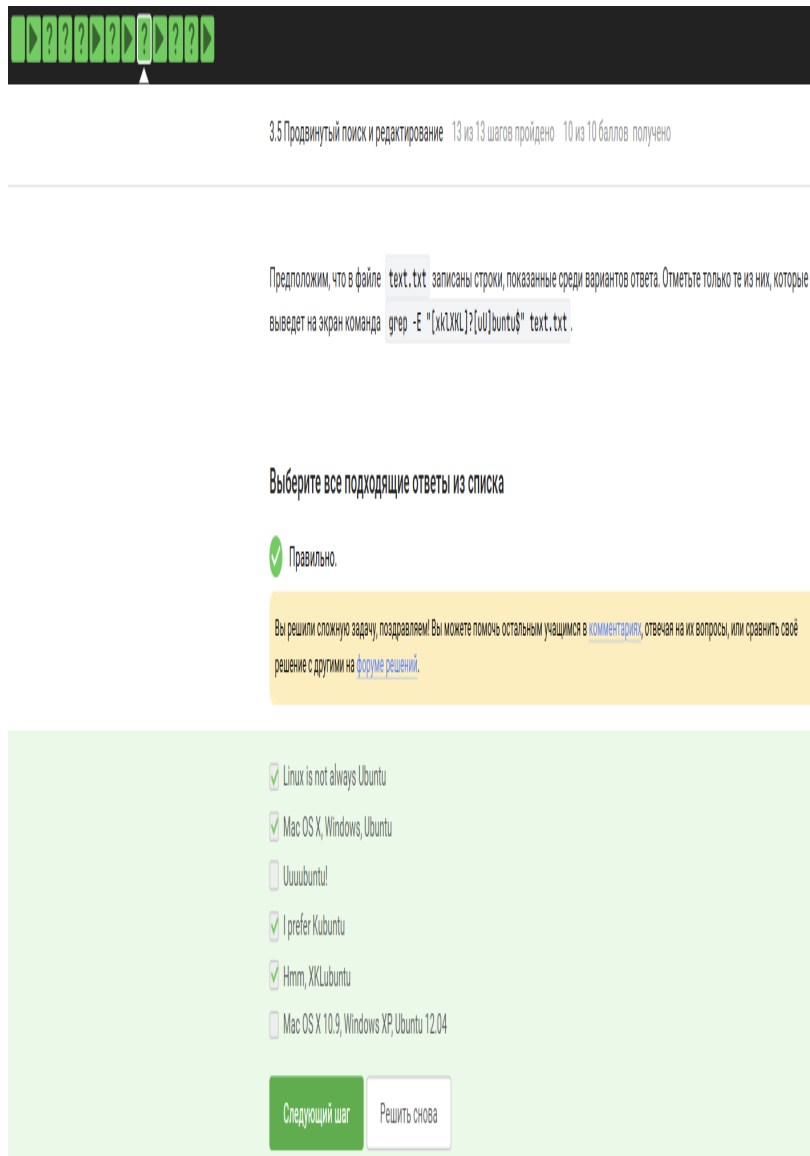


Рисунок 2.22: 22

Вопрос 6: sed без опции -n **Правильный ответ (отмечен ☒):** Каждая строка будет выведена два раза

Без -n sed печатает каждую строку (по умолчанию) и дополнительно печатает строки, попавшие под команду р.



3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Что произойдет, если в команде `sed -n '/[a-z]*/p' text.txt` не указывать опцию `-n`?

Выберите один вариант из списка

☒ Прекрасный ответ.

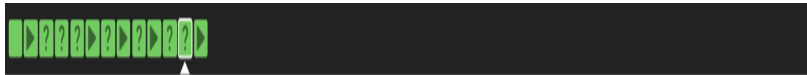
- ☐ Появится сообщение об ошибке
- ☐ На экран ничего не напечатается
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☒ Каждая строчка будет выведена два раза

Следующий шаг

Решить снова

Рисунок 2.23: 23

Вопрос 7 (текст задания): Замена аббревиатур (две и более заглавные буквы, окружённые пробелами) на «abbreviation» Задача решается с помощью `sed` и регулярного выражения (на скриншоте сам ответ не показан, но структура указана).



Под 'аббревиатурой' будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две 'аббревиатуры' подряд**. Например, текст " YOU YOU and YOU!" является **некорректным** (в нем есть две 'аббревиатуры', но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст "Hi, I heard these songs by ABBA, TLA and DM!", то он должен быть преобразован в "Hi, I heard these songs by ABBA, abbreviation and abbreviation!".

Примечание: после вашей замены 'аббревиатуры' на слово 'abbreviation' количество пробелов в тексте **должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с 'аббревиатурами' (это небезопасно, можно же ввести `rm -rf /*`). Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на 'abbreviation') на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная 'несовместимость' заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. *, +, ? и {}). Однако эту 'несовместимость' легко исправить указав при помощи {} какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве 'a') нужно записать как `(a){1,}` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

✓ Верно.

```
sed 's/[A-Z][A-Z][A-Z][A-Z]*/abbreviation/g' input.txt > edited.txt
```

Рисунок 2.24: 24

2.6 3.6 Строим графики в gnuplot

Вопрос 1: Опция gnuplot, чтобы графики не закрывались после выхода **Правильный ответ (отмечен ☐):** -p, -persist



3.6 Строим графики в `gplot` 8 из 10 шагов пройдено 2 из 7 баллов получено

Вы можете скачать и попробовать применить `gplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

✓ Правильно, молодец!

☐ `-raise`

☐ `-s, -show-plots-after-exit`

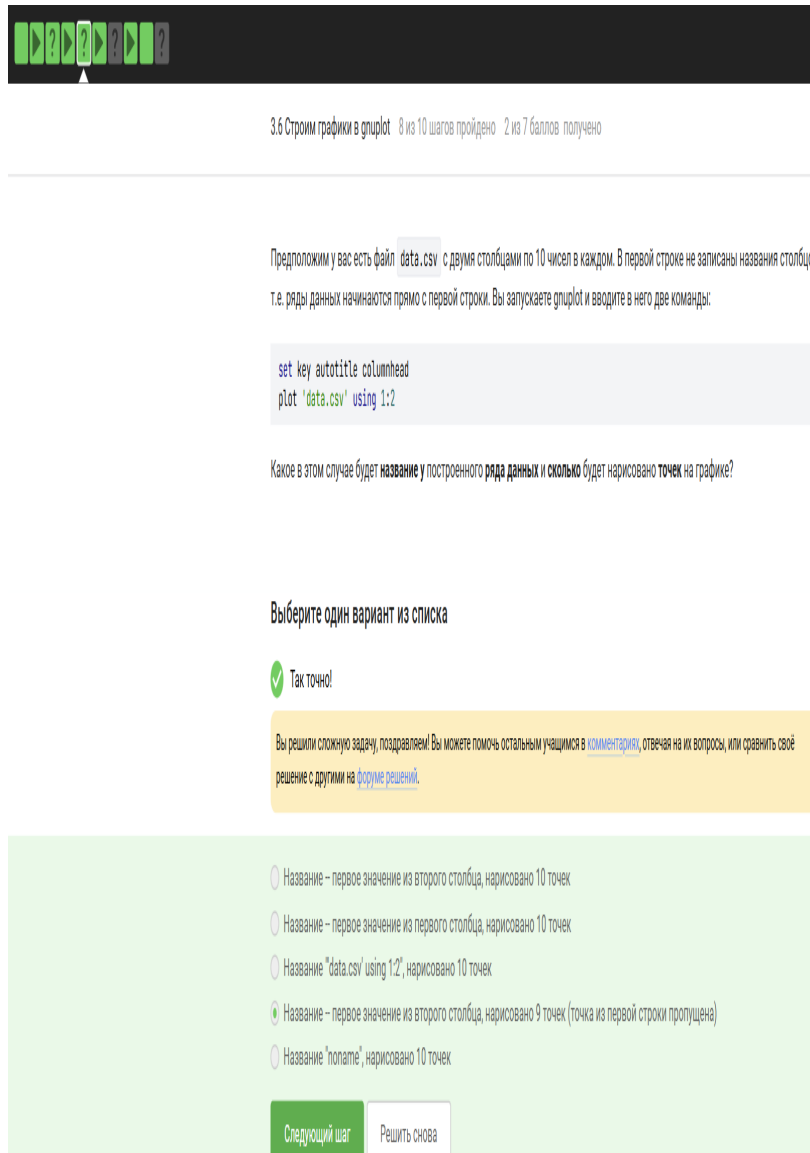
☒ `-p, -persist`

☐ Такой опции не существует

Рисунок 2.25: 25

Вопрос 2: `set key autotitle columnhead` и `plot „data.csv“ using 1:2` (нет заголовков) **Правильный ответ (отмечен □):** Название «`no name`», нарисовано 10 точек

`columnhead` использует первую строку как заголовок, но в файле нет заголовков, поэтому название ряда `no name`.



3.6 Строим графики в `gnuplot` 8 из 10 шагов пройдено 2 из 7 баллов получено

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного ряда данных и сколько будет нарисовано **точек** на графике?

Выберите один вариант из списка

✓ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☐ Название "data.csv/using 1.2", нарисовано 10 точек
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название "полете", нарисовано 10 точек

Следующий шаг

[Решить снова](#)

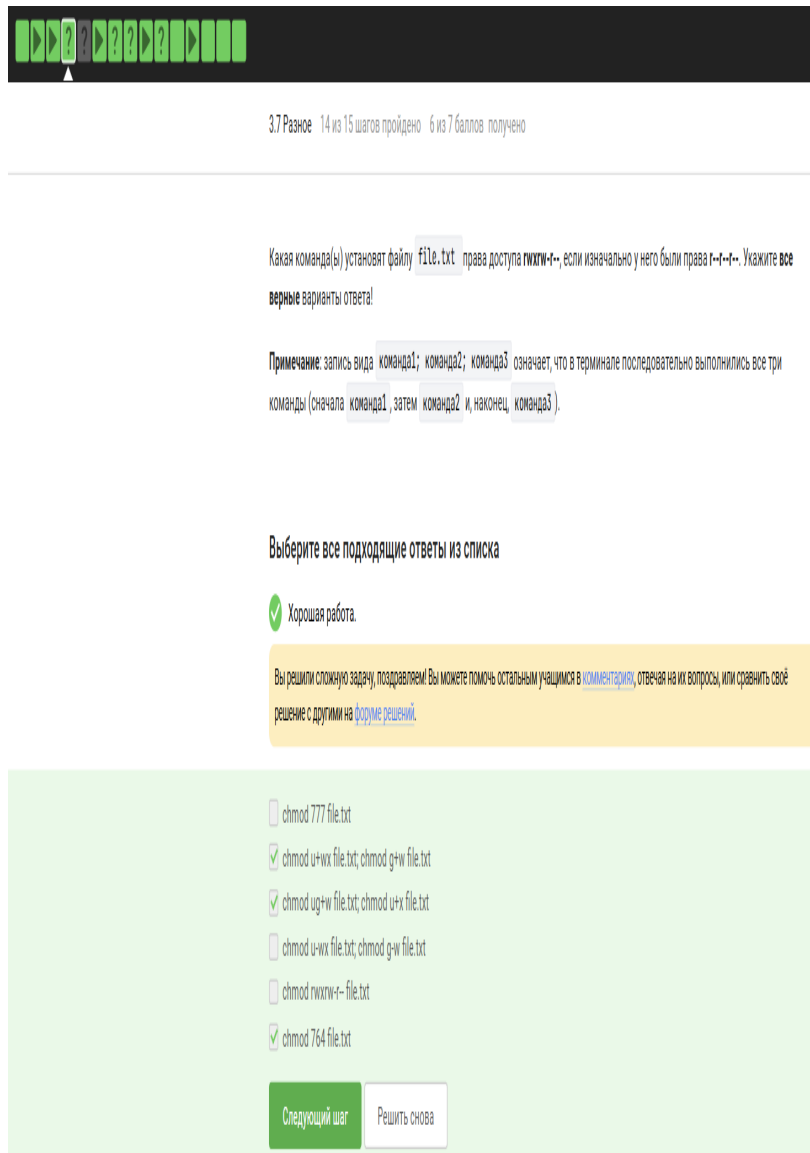
Рисунок 2.26: 26

2.7 3.7 Разное

Вопрос 1: Установка прав `rwXr-xr-` из `r-r-r-` **Правильные варианты (отмечены ☐):**

- `chmod ug+w file.txt; chmod u+x file.txt`

- `chmod 764 file.txt` (764 = `rwxrw-r--`)
- `chmod 777` — слишком много. `chmod u+xw file.txt`; - `chmod g+w file.txt` — неверно (группе нужен `r-x`, а не `rw-`).



3.7 Разное 14 из 15 шагов пройдено 6 из 7 баллов получено

Какая команда(ы) установят файлу `file.txt` права доступа `rwxrw-r--`, если изначально у него были права `r--r--r--`. Укажите все верные варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнялись все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod 777 file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☒ `chmod ug+w file.txt; chmod u+x file.txt`
- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☐ `chmod rwxrw-r-- file.txt`
- ☒ `chmod 764 file.txt`

Следующий шаг Решить снова

Рисунок 2.27: 27

Вопрос 2: Что можно посчитать командой `wc`? **Правильные (отмечены ☑):**

- Количество символов

- Количество строк

wc не считает буквы по отдельности, не считает предложения, размер файла в байтах (это wc -с — символы и байты совпадают в ASCII, но формально байты — тоже да, на скриншоте отмечены два варианта).

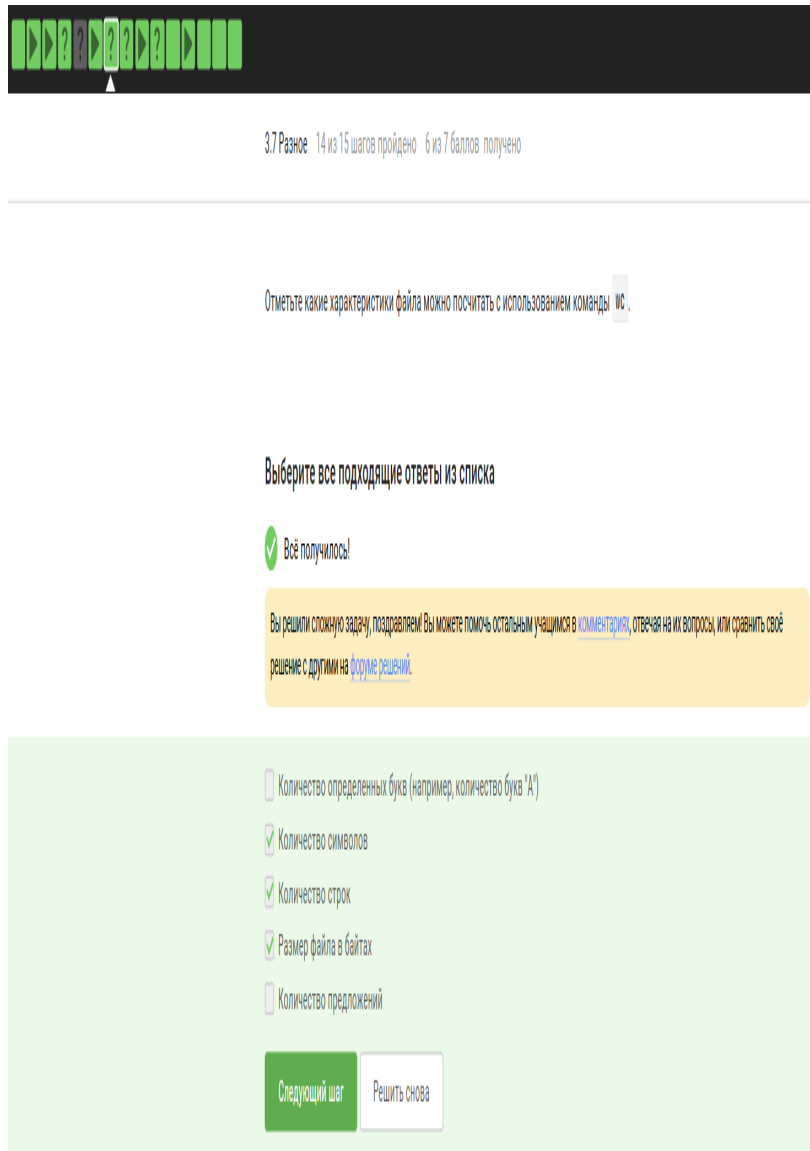
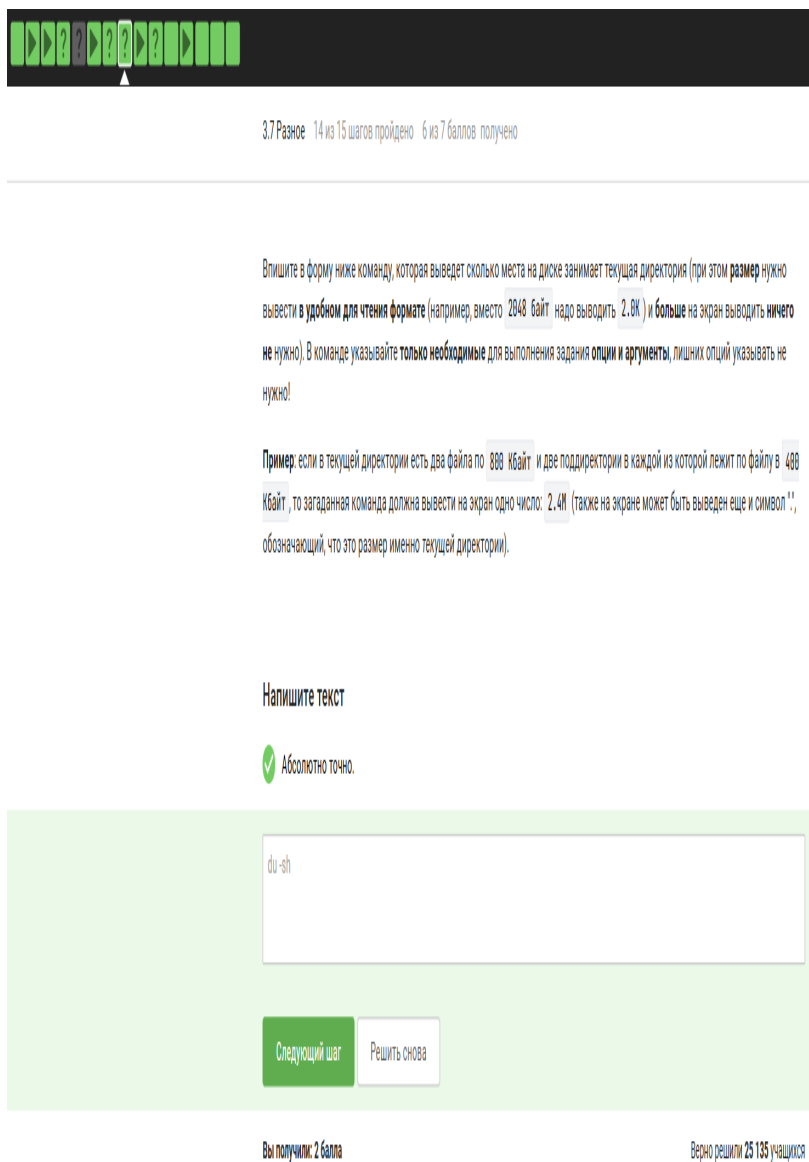


Рисунок 2.28: 28

Вопрос 3: Команда для размера текущей директории в удобном формате
Правильный ответ: `du -sh`

-s (суммарно), -h (human-readable).



3.7Разное 14 из 15 шагов пройдено 6 из 7 баллов получено

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2048 байт надо вывести 2.0K) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то заданная команда должна вывести на экран одно число: 2.4M (также на экране может быть выведен еще и символ "", обозначающий, что это размер именно текущей директории).

Напишите текст

✔ Абсолютно точно.

du -sh

Следующий шаг Решить снова

Вы получили: 2 балла Верно решили 25 135 учащихся

Рисунок 2.29: 29

Вопрос 4: Максимально короткая команда создания трёх поддиректорий

Правильный ответ: mkdir dir{1,2,3}

Фигурные скобки раскрываются в dir1 dir2 dir3.



3.7 Разное 14 из 15 шагов пройдено 6 из 7 баллов получено

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Хорошая работа.

`mkdir dir{1,2,3}`

Рисунок 2.30: 30

3 Заключение

Выполнены все задания по 3 разделу курса. Освоены:

- текстовый редактор vim (навигация, замена, выход);
- написание скриптов на bash: переменные, условия, циклы, функции, арифметика, обработка ввода;
- продвинутый поиск (find, grep, регулярные выражения) и редактирование (sed);
- построение графиков в gnuplot;
- управление правами доступа (chmod), анализ размера директорий (du), массовое создание каталогов. Напиши в формате маркдаун